Héritage et polymorphisme en Swift

420-249-AH - Programmation mobile iOS

1. Héritage

L'héritage permet de créer une nouvelle classe à partir d'une classe existante.

La nouvelle classe récupère les propriétés et les méthodes de la classe parente.

```
Appareil
   ↑
   |
Telephone
```

On peut ainsi réutiliser du code commun et spécialiser certains comportements.

2. Classe de base

Prenons une classe générale :

```swift
class Appareil {
    var nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }

    func afficherEtat() {
        print("\(nom) : \(batterie)%")
    }
}
```

Cette classe décrit les caractéristiques communes à plusieurs appareils.

3. Créer une classe dérivée

En Swift, l'héritage utilise :

```
:
```

Exemple :

```swift
class Telephone: Appareil {
}
```

Telephone hérite maintenant de Appareil.

4. Propriétés héritées

Même si Telephone ne déclare aucune propriété :

```
class Telephone: Appareil {
}
```

il possède déjà nom et batterie, car ces propriétés sont héritées de Appareil.

5. Méthodes héritées

Telephone hérite également des méthodes.

```
let telephone = Telephone(
    nom: "iPhone",
    batterie: 85
)
telephone.afficherEtat()
```

Résultat :

```
Console
iPhone : 85%
```

La méthode appartient initialement à Appareil, mais elle est disponible dans Telephone.

6. Ajouter des propriétés spécifiques

Une classe dérivée peut ajouter ses propres propriétés.

```
class Telephone: Appareil {
    var numeroTelephone: String = ""
}
```

Un Telephone possède maintenant :

```
nom             ← hérité
batterie        ← hérité
numeroTelephone ← spécifique
```

7. Ajouter des méthodes spécifiques

```
class Telephone: Appareil {
    var numeroTelephone: String = ""

    func appeler(numero: String) {
        print("Appel vers \(numero)")
    }
}
```

Utilisation :

```
telephone.afficherEtat()
telephone.appeler(numero: "514-555-1234")
```

8. Initialisation d'une classe dérivée

Supposons :

```
class Appareil {
    var nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }
}
```

La classe dérivée peut définir son propre initialisateur :

```
class Telephone: Appareil {
    var numeroTelephone: String

    init(
        nom: String,
        batterie: Int,
        numeroTelephone: String
    ) {
        self.numeroTelephone = numeroTelephone

        super.init(
            nom: nom,
            batterie: batterie
        )
    }
}
```

9. Le mot-clé super

super permet d'accéder à la classe parente.

Dans :

```
super.init(
    nom: nom,
    batterie: batterie
)
```

on demande à la classe Appareil d'initialiser ses propres propriétés.

On peut représenter l'initialisation ainsi :

```
Telephone
   │
   ├── initialise numeroTelephone
   │
   ↓
super.init(...)
   │
   ↓
Appareil initialise
nom et batterie
```

10. Exemple de création

```swift
let telephone = Telephone(
    nom: "iPhone",
    batterie: 90,
    numeroTelephone: "514-555-1234"
)
```

L'objet contient :

```
nom = iPhone
batterie = 90
numeroTelephone = 514-555-1234
```

11. Redéfinir une méthode

Une classe dérivée peut modifier le comportement d'une méthode héritée.

Swift utilise :

```swift
override
```

Exemple :

```swift
class Appareil {
    func afficherType() {
        print("Appareil")
    }
}
```

Puis :

```swift
class Telephone: Appareil {
    override func afficherType() {
        print("Téléphone")
    }
}
```

12. Utilisation de override

```
let appareil = Appareil()
let telephone = Telephone()

appareil.afficherType()
telephone.afficherType()
```

Résultat :

```
Console
Appareil
Téléphone
```

La même méthode possède maintenant un comportement différent selon l'objet.

13. Appeler la version du parent

Une méthode redéfinie peut également utiliser :

```
super
```

Exemple :

```
class Telephone: Appareil {
    override func afficherEtat() {
        super.afficherEtat()
        print("Type : Téléphone")
    }
}
```

Ici, super.afficherEtat() exécute d'abord la version définie dans Appareil.

14. Exemple — Classe parent et classe enfant complètes

Classe parent :

```
class Appareil {
    var nom: String

    init(nom: String) {
        self.nom = nom
    }

    func afficherInformations() {
        print("Nom : \(nom)")
    }
}
```

Classe enfant :

```swift
class Telephone: Appareil {
    var numero: String

    init(nom: String, numero: String) {
        self.numero = numero
        super.init(nom: nom)
    }

    override func afficherInformations() {
        super.afficherInformations()
        print("Numéro : \(numero)")
    }
}
```

15. Héritage sur plusieurs niveaux

Une classe peut hériter d'une classe qui hérite elle-même d'une autre classe.

```
Appareil
   ↑
Telephone
   ↑
TelephoneIntelligent
```

Exemple :

```swift
class TelephoneIntelligent: Telephone {
    var systeme: String = "iOS"
}
```

Il récupère alors les membres de Appareil et Telephone. Il faut cependant éviter les hiérarchies inutilement complexes.

16. Empêcher la redéfinition avec final

On peut empêcher une méthode d'être redéfinie :

```swift
final func numeroSerie() {
    print("123456")
}
```

Une classe enfant ne pourra pas utiliser override sur cette méthode.

17. Empêcher l'héritage

On peut également déclarer toute une classe :

```swift
final class Configuration {
}
```

Il sera alors impossible de créer :

```
class AutreConfiguration: Configuration {
}
```

18. Le polymorphisme

Le polymorphisme permet de manipuler différents objets à travers un type commun.

Supposons :

```
Appareil
   │
   ├── Telephone
   ├── Tablette
   └── Montre
```

Toutes ces classes sont des Appareil.

19. Exemple — Sous-classes de Appareil

```
class Appareil {
    func afficherType() {
        print("Appareil")
    }
}
```

Puis :

```
class Telephone: Appareil {
    override func afficherType() {
        print("Téléphone")
    }
}
```

Et :

```
class Tablette: Appareil {
    override func afficherType() {
        print("Tablette")
    }
}
```

20. Un type parent peut référencer un objet enfant

Ceci est valide :

```
let appareil: Appareil = Telephone()
```

Le type de la variable est Appareil, mais l'objet réel est Telephone.

21. Comportement polymorphique

```
let appareil: Appareil = Telephone()
appareil.afficherType()
```

Résultat :

```
Console
Téléphone
```

Swift utilise la version correspondant à l'objet réel. C'est le principe du polymorphisme.

22. Collection polymorphique

On peut maintenant créer :

```
let appareils: [Appareil] = [
    Appareil(),
    Telephone(),
    Tablette()
]
```

Puis :

```
for appareil in appareils {
    appareil.afficherType()
}
```

Résultat :

```
Console
Appareil
Téléphone
Tablette
```

23. Pourquoi est-ce intéressant ?

Sans polymorphisme, on pourrait être tenté de gérer séparément un tableau de téléphones, un tableau de tablettes, un tableau de montres.

Avec le polymorphisme :

```
[Appareil]
```

peut contenir tous ces objets. Le code devient plus général.

24. Exemple — Notification de base

```
class Notification {
    var titre: String

    init(titre: String) {
        self.titre = titre
    }

    func envoyer() {
        print("Envoi d'une notification")
    }
}
```

25. Notifications spécialisées

```
class NotificationMessage: Notification {
    override func envoyer() {
        print("Nouveau message : \(titre)")
    }
}

class NotificationBatterie: Notification {
    override func envoyer() {
        print("Alerte batterie : \(titre)")
    }
}
```

26. Utilisation polymorphique

```
let notifications: [Notification] = [
    NotificationMessage(
        titre: "Message de Marie"
    ),
    NotificationBatterie(
        titre: "Batterie à 15%"
    )
]
```

Puis :

```
for notification in notifications {
    notification.envoyer()
}
```

Résultat :

```
Console
Nouveau message : Message de Marie
Alerte batterie : Batterie à 15%
```

Même appel — notification.envoyer() — mais comportements différents.

27. Vérifier le type avec is

Swift permet de vérifier le type réel d'un objet avec :

```
is
```

Exemple :

```
if appareil is Telephone {
    print("C'est un téléphone")
}
```

is retourne true ou false.

28. Conversion de type avec as?

Supposons :

```
let appareil: Appareil = Telephone()
```

Le type déclaré est Appareil. On ne peut donc pas accéder directement aux membres spécifiques de Telephone.

On peut utiliser :

```
as?
```

29. Exemple avec as?

```
if let telephone = appareil as? Telephone {
    telephone.appeler(
        numero: "514-555-1234"
    )
}
```

as? effectue une conversion sécuritaire. La conversion retourne un optionnel.

30. Pourquoi as? retourne un optionnel ?

Parce que la conversion peut échouer.

Exemple :

```
let appareil: Appareil = Tablette()
```

Puis :

```swift
let telephone =
    appareil as? Telephone
```

Ici, telephone = nil, car une tablette n'est pas un téléphone.

31. as!

Swift permet également :

```swift
as!
```

Exemple :

```swift
let telephone =
    appareil as! Telephone
```

Mais cette conversion suppose que le type est obligatoirement correct.

Si ce n'est pas le cas : crash. On privilégie donc généralement as? lorsqu'il existe une possibilité d'échec.

32. Exemple avec une collection

```swift
for appareil in appareils {
    appareil.afficherType()

    if let telephone =
        appareil as? Telephone {
        telephone.appeler(
            numero: "514-555-1234"
        )
    }
}
```

On peut donc traiter tous les appareils, puis appliquer un comportement spécifique à certains types.

33. is vs as?

Outil	Utilisation
`is`	Demander simplement : est-ce ce type ?
`as?`	Manipuler ensuite l'objet comme ce type

Exemple avec is :

```swift
if appareil is Telephone {
}
```

Exemple avec as? :

```
if let telephone =
    appareil as? Telephone {
    telephone.appeler(...)
}
```

34. Héritage et polymorphisme ensemble

L'héritage crée la relation :

- Telephone est un Appareil
- Tablette est un Appareil
- Montre est un Appareil

Le polymorphisme permet ensuite de manipuler ces différents objets comme des Appareil, tout en conservant leur comportement spécifique.

35. Exemple guidé — Appareils mobiles

Créer :

```
class Appareil
```

avec :

```
nom
batterie
```

et :

```
func afficherInformations()
```

Créer ensuite :

```
class Telephone: Appareil
```

avec :

```
numeroTelephone
```

et redéfinir afficherInformations().

Créer :

```
class Tablette: Appareil
```

avec :

```
tailleEcran
```

et redéfinir également afficherInformations().

36. Créer une collection polymorphique

Créer :

```
let appareils: [Appareil] = [
    ...
]
```

contenant : un Appareil, un Telephone, une Tablette.

Parcourir le tableau et appeler afficherInformations(). Observer le comportement obtenu pour chaque objet.

37. Exercice — Système de notifications

Créer une classe de base :

```
class Notification
```

avec :

```
titre
message
```

et une méthode :

```
func envoyer()
```

Créer trois classes dérivées :

```
NotificationMessage
NotificationBatterie
NotificationSysteme
```

Chaque classe doit redéfinir envoyer() avec un message différent.

Créer ensuite :

```
let notifications: [Notification]
```

contenant plusieurs types de notifications. Parcourir le tableau :

```
for notification in notifications {
    notification.envoyer()
}
```

38. Exercice — Détection d'une notification critique

Pendant le parcours, utiliser :

```
is
```

pour détecter une NotificationBatterie et afficher :

```
Console
Une notification critique a été détectée.
```

Puis utiliser :

```
as?
```

pour convertir une notification dans son type spécifique lorsque nécessaire.

39. À retenir

L'héritage se déclare avec :

```
class Telephone: Appareil {
}
```

Une classe enfant hérite des propriétés et méthodes du parent.

Mot-clé	Rôle
`override`	Redéfinir un comportement
`super`	Accéder au parent
`final`	Empêcher l'héritage ou la redéfinition

Le polymorphisme permet :

```
let appareil: Appareil = Telephone()
```

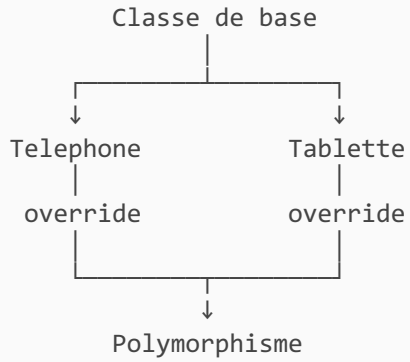
et :

```
let appareils: [Appareil] = [
    Telephone(),
    Tablette()
]
```

Le comportement exécuté dépend de l'objet réel.

Pour travailler avec le type réel	Effet
`is`	Permet de vérifier le type
`as?`	Permet une conversion sécuritaire
`as!`	Force une conversion et peut provoquer un crash

40. Vision globale



La prochaine étape sera particulièrement importante en Swift : Protocoles — définir un contrat sans imposer une relation d'héritage, avec protocol, puis comment une classe ou une structure peut adopter un ou plusieurs protocoles.